

SQL QUERIES USED FOR CREATING SALES ANALYSIS DASHBOARD

```
use classicmodels;

show tables;

/*select
customerNumber, customerName as company, country , creditLimit,
salesRepEmployeeNumber
from customers*/

select count(*) from customers

where salesRepEmployeeNumber is null;

-- order by two times are not allowed. we have to order the columns in the single order.
-- select * top 5 from customers;
-- where country = "USA";
-- select customerNumber, customerName , country, creditLimit from customers;
-- where (country = "germany" and creditLimit = 0) or (country = "USA" and creditLimit = 0);
-- where country in ("USA" ,"germany" ,"France")
-- where customerNumber in( 103,125, 119)
-- where creditLimit between 30000 and 80000
-- where customerName like '__o%'
-- from
-- where
-- select
-- orderby
-- select count(*) from customer;
-- select count(*) from customers
-- select count(distinct country) from customers

WINDOW FUNCTIONS.

/* select customerNumber,
customerName ,
country,
```

```

creditLimit ,
-- left(customerName,4) as shortName
-- length (customerName)
-- right(customerName,4) as shortName
-- repeat(customerName,3)
from customers;*/
-- select
-- orderNumber,orderDate,shippedDate,status
-- year(orderDate) -- year function date me se yaer bhahar nukal deta haa.
-- month(orderDate)
-- day(orderDate)
-- dayname(orderDate)
-- datediff( shippedDate,orderDate) as Daydifference
-- (Daydifference / 30.436875E) as MonthDifference
-- timestampdiff(MONTH, shippedDate,orderDate) as Daydifference
-- where orderDate = '2004-09-11' ;
-- from orders ;

```

```

select *, row_number() over( partition by orderNumber ) from orderdetails;
-- if any item change then the counting restart
-- agar name same haa to numbering increase hoti jaegi agar nhi to 1 se start ho jaegi
-- row number is used to add the new row.
-- partition reset the code as restart from the 1

```

```

select *, row_number() over( partition by productcode ) from orderdetails;
select * , dense_rank() over(order by ordernumber) from orderdetails;
-- ordernumber same ranking is same
-- order number change then the ranking change.
select *, rank() over(order by ordernumber) from orderdetails;
select *, rank() over(partition by orderNumber order by productCode) from orderDetails;

```

```

/*select *,
min(priceEach) over(),
avg(priceEach) over()
from orderdetails;*/

-- select *,
-- max(quantityOrdered) over( partition by ordernumber order by productCode) from
orderdetails;

select customerNumber,contactFirstName,country, lead(contactFirstName) over() from
customers;

select customerNumber,contactFirstName,country, lead(contactFirstName,2) over() from
customers;

select customerNumber,contactFirstName,country, lag(contactFirstName) over() from
customers;


-- aggregate functions.

-- select count(customerNumber,customerName,
country,salesRepEmployeeNumber,creditLimit) from customers;

-- aggregate functions means large input and give the small output.
select customerNumber,
-- min, max, avg, std,sum aggregate functions.
min(creditLimit) as Minimum,
max(creditLimit) as Maximum,
avg(creditLimit) as Average,
std(creditLimit) as standard_Deviation ,
sum(creditLimit) as Sum
from customers
group by customerNumber;


SELECT salesRepEmployeeNumber, COUNT(customerNumber) AS customerCount
FROM customers
GROUP BY salesRepEmployeeNumber
ORDER BY customerCount DESC;

```

```
select country,state ,  
sum(creditLimit)  
from customers  
group by country, state  
order by country;
```

```
select orderNumber,  
sum(QuantityOrdered) as Quantity  
from orderdetails  
group by orderNumber;
```

```
select orderNumber ,  
sum(priceEach * QuantityOrdered) as Total  
from orderDetails  
group by orderNumber;  
-- Data Filter
```

```
select country,sum(creditLimit)  
from customers  
group by country  
having sum(creditLimit) > 150000;  
-- aggregate function of filter krna hot haa to hum having use krte haa.  
-- count,min,max,avg,sum,std  
select * from Employees;  
-- column filter where and the aggregate filter on the having.  
select * from Employees  
where email like 'abow%';  
select count(*) from Employees;  
select officeCode ,count(employeeNumber)
```

```
from Employees
group by officeCode;
```

```
SELECT FirstName, LastName, COUNT(reportsTo) AS reportCount
FROM Employees
GROUP BY FirstName, LastName
ORDER BY reportCount DESC;
```

-- kosis kijiye ki count hamesha primary key hi kre.

```
select count( employeeNumber), reportsTo
from employees
group by reportsTo;
```

```
select * from Payments ;
select year(paymentDate) , customerNumber,sum(amount)
from payments
group by year(paymentDate),customerNumber
order by year(paymentDate);
```

```
select
customerNumber,
customerName,
contactFirstName,
salesRepEmployeeNumber,
creditLimit
from customers
where country = 'USA';
```

```
use pizzahut;
select
customers.customerName,
```

```

customers.customerNumber,
customers.contactFirstName,
customers.creditLimit
from classicmodels.customers;
-- dot ki help se navigte database column table inside that database.
-- dure database eme ja ker wohi database ko run krna.
-- id ,company,Name, CreditLimit.

-- select *into #temp1 from customers
-- not a number wali cheeje ko dimensions kehte haa.
-- does aggregation makes sense.
-- jiska su koi sense nhi bnat ato hum use dimension me rakh dete haa
-- jiska koi sense bna raha hot ahaa uske hun fact kehte haa.
-- marks,height, weight ,age,
-- Facts are numerical measures, and dimensions describe the attributes associated with those
measures

-- ek bar me sara data to use kehte haa pull load and
-- little data when we insert then it calles as the incremental load.
-- data migration by the two types- full load and the incremental load.
create table customers
(id int,company varchar(255), name varchar(255), creditLimit int);
insert into customers
select
customerNumber,
customerName,
contactFirstName,
creditLimit
from classicmodels.customers

```

```
where country = 'USA';
```

```
select * from customers;
```

```
use new_employee;
```

```
select
```

```
customers.customerName,
```

```
customers.customerNumber,
```

```
customers.contactFirstName,
```

```
customers.creditLimit
```

```
from classicmodels.customers;
```

```
create table Halo
```

```
(id int,company varchar(255), name varchar(255), creditLimit int);
```

```
insert into Halo
```

```
select
```

```
customerNumber,
```

```
customerName,
```

```
contactFirstName,
```

```
creditLimit
```

```
from classicmodels.customers
```

```
where country = 'USA';
```

```
select * from Halo;
```

```
use netmax;
```

```
-- dml commands data manipulation language select,insert,delete operations
```

```
-- ddl commands data defination language when we explain the structure of the table. creat,  
drop,alter,
```

```
-- data control language grant and revoke.
```

```
-- transaction control language.
```

```
select * from customers;
```

```
create table dummy( id int ,name varchar(255),marks int);
```

```
insert into dummy( id,name,marks)
```

```
values
```

```
(1,"Shubham",50),
```

```
(2,"Shubham",100),
```

```
(2,"shubham",300),
```

```
(2,"Abhi",400),
```

```
(3,"Akash",500),
```

```
(4,"Singh",500);
```

```
select * from dummy;
```

```
update dummy
```

```
set marks=500
```

```
where name='Shubham';
```

```
delete from dummy
```

```
where id=1;
```

```
delete from dummy
```

```
where id=2 and marks=100;
```

```
-- delete statement only delete the structure
```

```
truncate table dummy;
```

```
-- DELETE remove the specific row based on the given condition,
```

```
-- TRUNCATE removes all the record from the table at once,
```

```
-- whereas the DROP command removes the table or databases and as well as the structure
```



```
select * from customers ;
```

-- -- there are the three types of the output as the series and the columns and the output with more than single number

SUBQUERY

```
select * from customers
```

```
where creditLimit = (select max(creditLimit )from customers);
```

-- it give me the bill number.

```
select orderNumber,sum(total) from
```

```
(select orderNumber,
```

```
QuantityOrdered * priceEach as total
```

```
from orderdetails) as bill
```

```
group by orderNumber;
```

-- it give the output as the table.

```
select distinct
```

```
-- customerNumber,
```

```
-- customerName,country,
```

```
salesRepEmployeeNumber
```

```
from customers;
```

-- subquery always in the brackets.

```
select * from employees
```

```
where employeeNumber in
```

```
(select distinct
```

```
salesRepEmployeeNumber
```

```
from customers where country = 'USA');
```

```
use home;
```

```
create table cost (name varchar(255),costprice int);
```

```
insert into cost ( name ,costprice)
```

```
values('shirts',100),
```

```
('Jeans',200),
```

```
('Googles',300)
```

```
select * from cost;
```

```
create table sales(name varchar(255),sellingPrice int);
```

```
insert into sales(name,sellingprice)
```

```
values('shirts',300),
```

```
('Jeans',800),
```

```
('Googles',1000)
```

```
select * from sales;
```

```
update sales
```

```
set name='T_shirts'
```

```
where sellingprice=1000;
```

```
-- it is the join
```

```
select * from cost c join sales s on c.name=s.name;
```

```
select name from cost union all select name from sales order by name;
```

```
select * from cost c right join sales s on c.name=s.name
```

union

```
select * from cost c left join sales s on c.name=s.name;
```

```
select * from customers c join employees e on  
c.salesRepEmployeeNumber=e.employeeNumber
```

```
select * from orders join customers on orders.customerNumber= customers.customerNumber
```

-- pehle table ki foreign key and dusre table ki primary key use krenge.

-- one to one matching.we have done so far

-- next is one to many and many to one.

```
select * from orderdetails o join products p on o.productCode = p.productCode
```

-- one to many -> detail

-- normalized table ko jorna ha apas me.

-- Joining of the multiple table.

```
select productName,sum(sp) as total from
```

```
(select distinct country
```

```
,c.customerNumber,
```

```
c.customerName,
```

```
o.orderNumber,
```

```
o.orderdate,
```

```
o.status,
```

```
od.quantityordered ,
```

```
od.productcode,
```

```
p.productName,
```

```
p.productScale,
```

```
p.MSRP*od.quantityOrdered as sp,
```

```
p.buyprice*od.quantityOrdered as cp,
```

```
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
```

```
from customers c join orders o on
```

```
c.customerNumber = o.customerNumber
join orderdetails as od
on od.orderNumber = o.orderNumber
join products p
on p.productCode = od.productCode
where o.status = 'shipped') as Customer_Analysis
group by productName
order by total desc
```

```
select year(orderdate) ,sum(sp) as total from
(select distinct country
,c.customerNumber,
c.customerName,
o.orderNumber,
o.orderdate,
o.status,
od.quantityordered ,
od.productcode,
p.productName,
p.productScale,
p.MSRP*od.quantityOrdered as sp,
p.buyprice*od.quantityOrdered as cp,
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
from customers c join orders o on
c.customerNumber = o.customerNumber
join orderdetails as od
on od.orderNumber = o.orderNumber
join products p
on p.productCode = od.productCode
where o.status = 'shipped') as Customer_Analysis
```

group by year(orderdate)

order by total desc

--

-- select * from orderdetails

-- select * from customers

-- select * from products

-- pehle table ki foreign key and dusre table ki primary key.

select *

from employees as e

left join employees as b

on e.reportsto=b.employeeNumber;

-- in cross join on not be used here.

-- select * from first table cross join another table

-- schema anything that explain the structure of the table ;

-- full,left,right,inner,cross,self,

select productName,sum(sp) as total from

(select distinct country

,c.customerNumber,

c.customerName,

o.orderNumber,

o.orderdate,

o.status,

od.quantityordered ,

od.productcode,

p.productName,

p.productScale,

p.MSRP*od.quantityOrdered as sp,

```

p.buyprice*od.quantityOrdered as cp,
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
from customers c join orders o on
c.customerNumber = o.customerNumber
join orderdetails as od
on od.ordernumber = o.ordernumber
join products p
on p.productCode = od.productCode
where o.status = 'shipped') as Customer_Analysis
group by productName
order by total desc

```

```

select year(orderdate) ,sum(sp) as total from
(select distinct country
,c.customerNumber,
c.customerName,
o.orderNumber,
o.orderdate,
o.status,
od.quantityordered ,
od.productcode,
p.productName,
p.productScale,
p.MSRP*od.quantityOrdered as sp,
p.buyprice*od.quantityOrdered as cp,
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
from customers c join orders o on
c.customerNumber = o.customerNumber
join orderdetails as od
on od.ordernumber = o.ordernumber

```

```
join products p
on p.productCode = od.productCode
where o.status = 'shipped') as Customer_Analysis
group by year(orderdate)
order by total desc
```

```
select
c.customerNumber,
c.customerName,
c.country,
o.orderNumber,
o.orderdate,
od.quantityOrdered,
od.productCode,
p.productName,
p.MSRP,
p.buyPrice,
p.MSRP * od.quantityOrdered as sp,
p.buyPrice * od.quantityOrdered as cp,
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
from
customers as c join orders as o
on o.customerNumber = c.customernumber
join orderdetails as od
on od.ordernumber = o.ordernumber
join products as p
on p.productCode = od.productCode

select
```

```

c.customerNumber,
c.customerName,
c.country,
o.orderNumber,
o.orderdate,
od.quantityOrdered,
od.productCode,
p.productName,
p.MSRP,
p.buyPrice,
p.MSRP * od.quantityOrdered as sp,
p.buyPrice * od.quantityOrdered as cp,
p.MSRP * od.quantityOrdered - p.buyPrice * od.quantityOrdered as profit
from
customers as c join orders as o
on o.customerNumber = c.customernumber
join orderdetails as od
on od.ordernumber = o.ordernumber
join products as p
on p.productCode = od.productCode

```

```

-- 1) subquery
-- 2) temporaary table'
-- 3) view
-- subquery tab tak inda rehti haa tab tak

```

```

-- *****CTE*****->

```

```

-- format as with as
-- LIFE-> exist till included in the existed query

```


- no storage -> not store anywhere
- consumes processor--> kyuki is ko bar bar run krna parta haa. kyuki queries bna ke join krna parta haa bar bar
- accesible by user.
- Breaking down complex queries into smaller, reusable components.
- Improving readability and modularity by separating the logic.
- Enabling recursive operations for hierarchical data.

-- *****

with

SalesEMPbyCountry as(

select salesRepEmployeeNumber,country, sum(creditLimit) as total from customers
group by country,salesRepEmployeeNumber)

,empdetails as

(select * from SalesEMPbyCountry join employees on salesRepEmployeeNumber=employeeNumber)

select * from empdetails

create temporary table CountryWiseAnanlysis as

select count(customerNumber) as totalCustomer,country, sum(creditLimit) as total from customers

group by country,salesRepEmployeeNumber;

select * from CountryWiseAnanlysis;

-- *****TEMPORARY
TABLE*****

-- last till session ends

-- eat storage

-- we can drop the temporary table manually

-- donot eat the processor

-- we can access the data fast from it
-- accessible by ourself not by the other user.
-- priority is high as if same name then temporary and table then the temporary table has high priority

-- *****VIEW

```
create view v_CountryWiseAnalysis as
select count(customerNumber) as totalCustomer,country, sum(creditLimit) as total from
customers
group by country,salesRepEmployeeNumber;
```

-- view data store nhi krta view query store krta haa.alter
-- lifetime is permanent
-- last till manually dropped
-- query is stored
-- consumption of the processor is much more as it consumes more processor.
-- accessible by everyone.

-- *****
TRANSACTIONS *****

```
create temporary table test(id int,name varchar(255));
```

-- checkpoint
start TRANSACTION;

```
insert into test
values(1,'Kirat'),
(2,'Abhi');
insert into test
values
(3,'Akash')
```

```
select * from test
-- all transaction revert back kr di
-- rollback pura pura ctrl z ki tarah kam krta haa
-- fully satisfied so we commit;
commit;
rollback ;
```

```
savepoint saveOne;
insert into test
values
(4,'Rahul');
```

```
rollback to saveOne;
insert into test
values
(5,'ABC');
```

```
rollback
-- Transaction issues
-- Dirty Read ->( reading uncommitted data)
-- non repeatable read->
-- phantom read->
-- view and the materialized view
-- clustered and non-clustered index.
-- data arranged in primary key called clustered index
-- non-clustered index -> create index on _____ column name _____
-- table remain same a new table generate so when we search then it search from the new
table generated.

-- shift alt down
-- functions in the sql.
```

-- stored procedure in the sql.

use netmax;

create table products(p_id int primary key auto_increment,name varchar(255),price int);

drop table orders;

create table orders(order_id int primary key auto_increment,quantity int, productkey int,

foreign key(productkey) references products(p_id)

on delete set null) ;

-- cascade means agr 3 no delete to poora ka poora record hi saaf krdo .

-- foreign key and the primary key.

insert into products(name,price)

values ('Chocolate',20),

('Apple',50),

('pen',10),

('balls',25);

select * from products;

-- foreign key pe unique constraint so that dobara forign key nhi aa sakti

-- it give error because we cannot add the data of the item no 5 as item no5 not exists.

insert into orders(quantity,productkey)

values(10,3)

(5,1),

(3,2),

(10,3);

select *from orders

-- 50 2 10 3 8 4

--

delete from products where p_id = ;

```
create table Students_1(id int primary key, name varchar(255),marks int);
```

```
insert into Students_1
```

```
values
```

```
(1,'Shubham',20),
```

```
(2,'abhi',50),
```

```
(3,'akash',40),
```

```
(4,'sakshi',10)
```

```
select * from Students_1
```

```
-- non clustered index
```

```
create index idx_marks on Students_1(marks);
```

```
drop table orders
```

```
use classicmodels
```

```
select * FROM customers where creditLimit>100000;
```

```
-- clustered arrange physiccaly
```

```
-- if there is any copy of the data then it is the non clustered index.
```

```
-- clustered
```

```
create index idx_creditLimit on customers(creditLimit);
```

```
-- use netmax
```

```
-- use classicmodels
```

```
select * from customers
```

```
where country='USA'
```

```
union all
```

```
select * from customers
```

```
where country ='France';
```

```
select * from orderdetails;
```

```
-- kuch code likha or manually execute krwate haa.
```

-- delimiter change give name begin then end

delimiter //

create procedure example()

begin

select * from customers;

end //

delimiter ;

call example();

drop procedure example;

-- *****

delimiter //

create procedure CheckCreditLimit(in id int)

begin

select customerName,creditLimit from customers where customerNumber = id;

end //

delimiter ;

-- to execute we have to call the function.

call CheckCreditLimit(114);

drop procedure CheckCreditLimit;

--

with orders as

(select

year(orderdate) as Year,

```
month(orderdate) as Month,  
sum(priceEach* quantityOrdered) as Sales  
from orders join orderdetails on  
orders.orderNumber = orderdetails.orderNumber  
group by year(orderdate),month(orderdate))  
select *,  
(sales-lag(sales) over(order by year,month) )/lag(sales) over(order by year,month) as Growth  
from orders;
```

```
-- monthly sales.  
-- growth final-initial/initial
```

```
delimiter //  
create procedure CompareSales()  
begin  
    with orders as  
(select  
year(orderdate) as Year,  
month(orderdate) as Month,  
sum(priceEach* quantityOrdered) as Sales  
from orders join orderdetails on  
orders.orderNumber = orderdetails.orderNumber  
group by year(orderdate),month(orderdate))  
select *,  
(sales-lag(sales) over(order by year,month) )/lag(sales) over(order by year,month) as Growth  
from orders;  
end //  
delimiter //
```

call CompareSales()

```
-- ***** TRIGGERS
*****
```

delimiter //

create procedure

begin

-- triggers automaticalyy work done we not suppose to be doen manually.

end;

delimiter ;

drop table orders_1;

create table orders_1(order_id int, Order_date Date ,Sdate Date default null);

-- jab bhi data search krna ho hum lookup table me dekhte haa

-- log table records rakhti haa kab kya huya

create table order_log(id int, status varchar(255));

alter table order_log

add column logtime datetime default now();

delimiter //

create trigger InsertOrder

after insert on orders_1

-- insert ke bad table me value pao

-- aftre update emans update ke bad value poadusre table me.

for each row

begin

insert into order_log(id,status) values (new.id,'In-progress');

end //

delimiter ;


```
-- for loop while loop .  
-- log table sirf insert  
delimiter //  
create trigger Insert  
-- old insert hone se pehle  
-- for purana data old.id
```